

答辩准备 — 基于个人开发日志的逐日问答

聂琦钢 | 学号 2024006850 | 日志覆盖 6.15 - 6.25

老师可能会翻开你的日志，指着某一天的记录问："这天你遇到了什么问题？怎么解决的？" 或 "AI 在这里面做了什么？" 以下按日期逐一准备。

6.15 — 选题与技术调研

老师可能问：

"你怎么确定选题的？为什么觉得现有工具不够好？"

答：我调研了 VisuAlgo、Algorithm Visualizer 等工具，发现四个共性问题——算法覆盖不全（通常只有 20-30 个）、没有代码同步高亮、用户不能自定义输入数据、都是网页版需要网络。所以我定位为"桌面端 + 55 个算法 + 代码同步 + 可自定义数据"，这几个差异化点从一开始就明确了。

老师可能问：

"为什么在 Electron 和 Tauri 之间选了 Electron？"

答：我做了对比分析。Tauri 更轻量（不打包 Chromium），但生态不如 Electron 成熟，M1 兼容性当时还有文档不完善。加上团队 5 个人都有 React 基础，Electron 的 Chromium 渲染天然兼容 Web 技术栈，学习成本最低。后来打包时遇到的网络问题也证实了 Tauri 可能也有类似问题——不是说选 Tauri 就能省事。

6.16 — 接口与架构设计

老师可能问：

"VisualStep 接口是怎么设计出来的？"

答：核心思路是"要能同时描述 55 个算法"。我从冒泡排序、链表遍历、Dijkstra 三个差异最大的算法入手，找出它们的共性：每一步都需要知道"数据现在是什么状态（data）"、"哪些元素在操作（activeIndices）"、"哪些已经完成了（markedIndices）"、"这是什么类型的操作（actionType）"。然后把这三个共性抽象为核心字段。扩展字段（graphEdges、gridData）留给图算法和网格算法按需使用。

老师可能问：

"你怎么平衡步骤粒度的粗细？"

答：如果太粗（比如一次把整轮冒泡合成一步），动画就没意义了——学生看不清楚每次比较和交换。如果太细（比如把 `i++` 也做成一步），步骤数会爆炸。我定的规则是：每个"有视觉意义的原子操作"作为一步——一次比较、一次交换、一次移动、一次松弛。这样冒泡排序约 45 步，Dijkstra 约 20 步，步骤数在 20-100 的合理范围。

老师可能问：

"AI 在这里面做了什么？"

答：我让 AI 对比了 Zustand、Redux、Context 三种方案的优劣，生成了对比表。我根据对比表做了最终决策。AI 没有替我做决定，但帮我整理了信息。

6.17 — 需求分析文档

老师可能问：

"需求分析花了多长时间？怎么做的？"

答：约一天半。分三个部分：问题定义（5 大教学痛点）、可行性研究（技术/经济/操作/法律四个维度）、需求分析（用例图、数据流图、E-R 图）。可行性研究的结论是"完全可行"——全部使用开源技术栈，零软件许可费，本地分发零运维成本。

老师可能问：

"Mermaid 图遇到什么问题？"

答：mindmap 图在 VS Code 不渲染，因为 mindmap 是 Mermaid 9.3+ 才支持的特性，而常见的 Markdown 预览插件用的是旧版。我改成了 `graph LR` 语法重新绘制，兼容性就好多了。后续还遇到了 stateDiagram-v2 报错、DFD 里 `[]` 被当节点形状等问题。这些都是 AI 生成后人工验证发现的——AI 生成的 Mermaid 语法不一定全部兼容，需要自己验证修正。

老师可能问：

"55 个算法清单怎么定出来的？"

答：我以王道考研数据结构和严蔚敏《数据结构》教材的目录为基准，对照了 LeetCode 热题 100 和 CS61B 的课程大纲，归纳出 7 大分类。每个分类下列出核心算法，最终确定了 55 个。AI 帮我扩展了清单，但我对照教材确认了覆盖范围。

6.18 — 核心代码实现（工作量最大的一天）

老师可能问：

"这一天完成了什么？"

答：这一天是工程搭建和核心实现最密集的一天。完成了项目脚手架（Electron + Vite + React + TS + Tailwind 全部配置）、类型系统（types/algo.ts, 15+ 接口）、Zustand 状态机（14 个 Action 方法）、前 6 个算法的步骤生成器、4 种 Visualizer 组件、7 个 UI 组件、Electron 主进程。当天 TypeScript 零错误，Vite 构建 300KB。

老师可能问：

"postcss 配置遇到什么问题？"

答：postcss.config.js 报了 `MODULE_TYPELESS_PACKAGE_JSON` 警告。原因是 package.json 没有 `"type": "module"` 字段，Node.js 不知道该文件是 CommonJS 还是 ES Module。解决方案是把配置文件后缀改为 .mjs——显式声明 ES Module 类型。同样 tailwind.config.js 也改成了 .mjs。这样消除了警告，也避免了性能开销。

老师可能问：

"Electron 安装失败怎么解决的？"

答：npm install electron 时报 socket hang up 和 ETIMEDOUT。原因是 Electron 的二进制包托管在 GitHub Releases 上，从国内直接下载经常超时。解决方案是设置 ELECTRON_MIRROR 环境变量，指向 npmmirror.com 的国内镜像。后来打包 Windows 时也是用这个镜像加速的。

老师可能问：

"ArrayVisualizer 的柱形图高度为什么用百分比不行？"

答：CSS 百分比高度需要父容器有明确高度。但柱子的父容器是 flex 容器，items-end 对齐，高度由内容撑开 (auto)。所以在 flex 容器里 height: 80% 是无效的。解决方案是给外层容器一个固定的像素高度 (320px)，然后每条柱子的高度用像素计算： $barH = (value / maxValue) \times 280$ ，保证 barH 在 6px-280px 之间。

老师可能问：

"这天 AI 帮你做了什么？"

答：量非常大。AI 生成了全部配置文件的框架、类型系统、状态机、6 个算法生成器、4 种可视化组件、7 个 UI 组件。总生成代码约 3000 行，我人工修改了约 20%。修改主要在柱形图的高度计算方案（AI 最初用的百分比，我发现不行后改成像素）、动画参数、以及代码格式统一。

6.22 — 系统设计与打包

老师可能问：

"系统设计文档包含哪些内容？"

答：总体设计（5 层架构图、8 项技术选型对比表、包图、部署架构）+ 详细设计（类图、时序图×3、状态图、接口设计、E-R 图、存储设计）。共 11 张 Mermaid 图。全部图表在 mermaid.live 上验证过兼容性。

老师可能问：

"哈夫曼树可视化怎么迭代的？"

答：经历了 4 版迭代，这个过程的日志里也写了。第一版均匀分布——偏斜树节点全堆在中间，完全不行。第二版用最终树结构做位置遮罩——但初始状态叶子在深层索引，内部的零值节点阻断了 TreeVisualizer 的 DFS 遍历，导致过程空白。第三版用"森林链式"——但索引冲突导致节点重叠。第四版最终采用"过程显示频率数组（确保一直有数据）+ 最后一步切换为完整树形"。虽然中间过程不是理想的渐进生长，但保证了全程可视化不空白。

老师可能问：

"electron-builder 版本号为什么要改？"

答：package.json 里写的是 "electron": "^30.5.1"，开头的 ^ 表示兼容 30.5.1 及以上的小版本。但 electron-builder 需要精确的版本号，因为不同版本对应不同的平台二进制包。报了 "version is a range, not a fixed version" 错误。把 ^ 去掉改成 "30.5.1" 就好了。electron-builder 也是同样的问题。

老师可能问：

"tsc 类型检查报错怎么解决的？"

答: `tsconfig.node.json` 里 `composite: true` 和 `declaration: false` 冲突——`composite` 项目必须输出声明文件。我把 `composite: true` 删掉了。因为 Electron 主进程的 `tsconfig` 只是一个独立编译配置, 不需要被其他项目引用。

6.23 — 算法补齐与图输入

老师可能问:

"图算法自定义输入是什么格式?"

答: 边列表格式 `[N, from, to, weight, ...]`。比如 `5, 0, 1, 4, 0, 2, 1, ...` 表示 5 个节点的图, 然后每 3 个数字一组表示一条边。Dijkstra、BFS、DFS、Prim、Kruskal 全部统一这个格式。相比旧的 `[0,1,2,3,4]` 格式 (只有节点编号), 新格式让用户可以完整自定义图结构。

老师可能问:

"字符直输怎么实现的?"

答: `DataControl` 的输入框里检测到非数字字符 (`/[^0-9,\s\-\]/`), 且当前算法是括号匹配或中缀转后缀时, 进入符号模式。每个字符转成 `charCodeAt(0)` 传给算法。柱形图上通过 `showChars` prop 控制是否显示 `String.fromCharCode(value)` 字符标签——避免排序算法里数字 42 被显示成 `'*'`。

老师可能问:

"邻接表从 grid 改回链表是为什么?"

答: 第一版用 grid 展示, 格子里存的是 `to × 100 + weight` 编码值 (比如 104 表示节点1权重4), 识别度太低。改成 linked-list 可视化, 5 条链表串联, 用 -1 分隔。每条链对应一个节点的所有邻居, 比 grid 直观得多。

6.24 — Windows 打包

老师可能问:

"Windows 打包怎么做的? Mac 上能打包 Windows 吗?"

答: `electron-builder` 支持跨平台打包, 但需要下载对应平台的 Electron 二进制和构建工具。Electron 主体用 `ELECTRON_MIRROR` 镜像下载成功了 (109MB, 耗时 35 秒)。但 `winCodeSign` (代码签名工具) 下载超时了 4 次, 第 5 次才成功。还有 `nsis-3.0.4.1.7z` (安装包制作工具) 也超时过。最终通过反复重试全部下载成功, 产出了 NSIS 安装包 (74MB exe) 和免安装版 (106MB zip)。

老师可能问:

"为什么不用 CI/CD 自动构建?"

答: 这个我们反思过了。如果提前配置 GitHub Actions, push 代码后自动触发构建, 就不会受本地网络影响。这是后续改进的一个方向。当时时间紧, 优先保证手动打包能成功。

6.25 — 最终文档与提交

老师可能问:

"文档用了什么工具写？"

答：Markdown 格式，用 Typora 和 VS Code 编写。Mermaid 图表在 mermaid.live 上验证。最终项目文档用 marked.js 转 HTML 再通过 macOS textutil 转成 .docx。这个转换过程最初有乱码问题——因为第一版用在线工具转了之后中文编码丢失。后来用 marked 正确设置 UTF-8 和字体后才正常。

老师可能问：

"日志里写的 AI 使用统计——60 轮对话、6000 行代码、20% 修改——这些数字怎么来的？"

答：对话轮次是估算的（回顾每次交互），代码行数是用 `wc -l` 统计了 AI 直接输出的代码块，修改比例是我自己估计的——大部分算法生成器我只改了边界条件和描述文字，大约 20%；可视化组件和动画参数改得多一些，约 25%；配置文件改得少，约 10%。

通用防御回答

如果老师挑你日志里某个具体问题追问，但你一时想不起细节：

"老师，这个问题我当时记录在日志里了，具体细节容我回忆一下... ..（停顿 3 秒）... ..是的，当时的情况是这样的... .."

要点：

1. 先承认问题发生过（不要否认）
2. 简述解决思路（不用逐字背日志）
3. 如果有的话，补充一句"这个问题后来我反思过，其实可以... .."（展示成长性）